
RLAR: Revised Reward Construction Methodology

Shared-prefix and efficient-call revision — September 23, 2026

1 Methodology

RLAR treats reward design as a bounded interaction with tools: an agent constructs an executable reward program, tests its behavior against task-provided evidence, and revises it before reuse. The output is a program and its validation record, rather than a score for one response. We distinguish construction tools, which help the agent design and test a program, from scoring components, which the resulting program invokes. A single controller coordinates construction; additional specialized agents are not required.

Scope of this revision. We specify a revised protocol for objectively verifiable tasks, initially mathematics and code. The construction-loop comparisons and Qwen3-8B distillation below are proposed evaluations, not completed experiments. Earlier selection and RL results do not establish their effectiveness; the selection-only study is retained in Appendix B. Extending validation to open-ended translation and dialogue requires independent quality labels and is outside this revision’s primary scope.

1.1 Reward Construction as a Task

A construction task τ provides a specification d_τ , permitted scoring resources $\mathcal{A}_\tau$, and two disjoint test suites V_τ^{dev} and V_τ^{audit} . The specification defines the objective, input schema, available references or execution tests, and any explicit output constraints, including their priority relative to content correctness. Each suite contains response examples, oracle-derived quality labels or comparisons, and valid transformations for checking reward behavior. These labels come from task verifiers fixed before construction, not from the reward designer’s own judgments. Development tests provide feedback; the audit suite remains inaccessible until the submitted program is frozen.

The agent produces a reward artifact g_τ with interface

$$g_\tau(q, r, \hat{r}, z) \longrightarrow (s, e), \quad s \in [0, 1], \quad (1)$$

where q is a query, r a response, $\hat{r}$ an optional training reference, and z the permitted task metadata or execution tests. The structured record e contains component scores and execution status. Infrastructure failures return a distinct error status rather than a valid low score. An artifact comprises source code, an applicability description, pinned dependencies, and fixed score transformations and composition parameters. References and tests are function inputs; programs may not embed development-example identifiers or answer lookup tables.

The harness consumes a stream of query records and emits one construction result per processed record. Each result stores a serialized reward definition with function-source strings, or a key into an optional content-addressed library. A key hashes the complete definition, including composition and runtime dependencies; query inputs and validation evidence are stored separately. A query without an adequate test pack can yield an unvalidated definition, but not a claim of validated reward quality.

Construction occurs at the level of a task specification or reusable task subtype. At scoring time, a router selects an applicable artifact from the library using the query and permitted metadata. It does not inspect the current candidate responses to choose their reward function. When no artifact covers the required contract, a bounded construction episode is triggered using the task’s development pack.

Table 1: Minimal construction interface. File and shell tools are optional and scoped to permitted resources. Versioning, validation, and registration remain harness responsibilities.

Tool	Input	Observation / effect
read_file	Path and bounded line range.	Content, file hash, and truncation status.
write_file	Path, content, expected prior hash.	Updated path and content hash.
edit_file	Path, prior hash, exact replacement.	Change record and updated hash; conflicts are explicit.
run_command	Command, working directory, timeout.	Exit code, bounded stdout/stderr, and termination reason.
test_reward	Serialized definition, development profile, on-pass policy.	Version-bound report; optional finalization after full admission checks.
submit_reward	Artifact ID and validation run ID.	Acceptance or rejection and selected artifact; the driver persists the result.

1.2 Modular Tools and a Fixed Reward-Model Catalogue

The stream interface requires only structured reward definitions, task-aware validation, and submission (Table 1). General file and shell operations are optional adapters for agents that benefit from a coding workspace. Task analysis is performed by the controller from the supplied task pack; model cards are ordinary read-only resources. Neither requires a separate LLM-backed tool. Existing coding-agent file and shell interfaces can be adapted to this schema. A dedicated registry search is optional for larger libraries; small catalogues can be read directly.

Fixed neural scoring resources. We replace online model search, checkpoint downloading, and automatic deployment with a small, preselected catalogue $\mathcal{M}_0$ of representative open-source reward models, exposed through a uniform `score_model` API. Each entry includes its model card and a versioned inference contract. The catalogue is selected before evaluation, without using target audit results, and shared by all construction baselines. Exact checkpoints and revisions belong in the experiment manifest. Inspecting a card suggests applicability; it does not certify reward quality. The agent can use these models as components, combine them with rules, or omit them. Catalogue membership stays fixed even as the library of constructed programs grows.

Single reward and checklist modes. The run configuration selects a single function or a checklist. In checklist mode, the agent specifies a bounded list of criteria and a source-code string for each component; the plan and all implementations can share one completion. The entire list and its aggregation form one artifact. Functions may implement answer checks, code execution against permitted tests, or calls to model APIs. A fixed runtime aggregates their normalized scores:

$$s = h_\tau(q, r, z) \frac{1}{|\mathcal{V}|} \sum_{j \in \mathcal{V}} c_j(f_j(q, r, \hat{r}, z)), \quad \mathcal{V} \neq \emptyset, \quad (2)$$

Here $\mathcal{V}$ is the set of successfully executed components. Single mode uses one function; checklist mode uses the arithmetic mean over successful components, with a bounded planned count m . All-component execution failure returns no score and a failed status. Coverage $|\mathcal{V}|/m$ and each component’s status accompany partial scores. This revision does not expose trainable or agent-selected weights. Each c_j maps its component to $[0, 1]$ with larger values indicating better quality. Component count is bounded, and normalization, criteria, and the component list are fixed before scoring a query’s candidates. The planned list is fixed, and membership in $\mathcal{V}$ is determined only by the declared execution-status rule, never by dropping low scores. Valid zero scores remain in the mean. Partial execution does not itself establish reward quality. The optional gate $h_\tau \in \{0, 1\}$ enforces requirements explicitly present in the task; it is 1 when no hard gate is required. Duplicate or overlapping criteria are diagnosed because repeated checks can implicitly alter emphasis even under equal averaging. Both component behavior and the final aggregate require validation; a list of executable checks alone does not establish reward quality.

1.3 A Bounded Construction and Repair Loop

An episode starts from the current query/task pack, a library snapshot, and an explicit resource budget B ; a writable workspace is optional. The controller inspects relevant contracts, retrieves

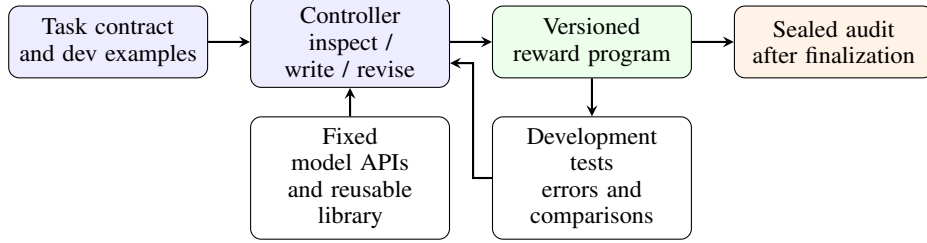


Figure 1: Revised construction loop. Development feedback supports repair; audit outcomes never return to the controller. Accepted artifacts accumulate in the library, while all interaction events are stored separately as traces.

reusable artifacts, writes or adapts a candidate, and calls `test_reward`. The harness snapshots the entire artifact before testing and binds its report to that immutable version. A failed comparison or execution error can lead to a targeted source revision and another test. Submission checks the matching full-development report; agent-written logs cannot certify acceptance. The interaction is

$$a_t \sim \pi_{\text{design}}(\cdot \mid C_t), \quad o_t = \text{Tool}(a_t), \quad (3)$$

where $C_t = P_{\text{run}} \parallel P_{\tau} \parallel h_t$ is the logical message sequence and $\parallel$ denotes concatenation. The run prefix fixes instructions and tool schemas; the episode prefix fixes the task pack, initial library snapshot, mode, and initial budget. History appends complete assistant actions, their tool observations, and updated state, including remaining budget B_t . Earlier messages are never rewritten. Each episode has at most one controller request in flight; independent tools may run as a batch, with results appended in call order before the next decision. Different queries have separate histories. This preserves a common prefix and permits provider-supported prefix caching without assuming a cache hit.

Reducing model requests. The controller can emit a plan and complete definition in one call. Tools perform lookup, hashing, batched testing, and persistence without auxiliary LLM planners or judges. Under the predeclared on-pass submission policy, `test_reward` invokes the same trusted finalizer as explicit submission after full admission checks, eliminating a confirmation-only model turn. Repairable failures trigger further decisions. Compatible validated reuse can require zero controller calls, and a new artifact that passes immediately can require one. These are execution paths, not measured speedups; retries and neural scoring requests are counted separately (Appendix A.3).

The budget limits controller tokens, candidate submissions, and test executions; their realized costs are reported separately. Finalization selects an eligible candidate or records failure when the budget is exhausted. A retained earlier candidate must pass the same development criteria. Failed episodes remain part of the evaluation denominator. Budgets cover all checklist items and retries jointly. A supervisor enforces call and episode deadlines, limits repeated invalid/no-progress actions, and never resets the full budget for each component. Context capacity is checked before requests; exhaustion follows the same termination rule rather than silently removing history or invoking an LLM summarizer.

The outer driver owns stream iteration and durable output. Record-specific failures produce explicit results and allow subsequent queries to proceed; authentication, output-storage, or persistent infrastructure failures stop the run. EOF, cancellation, and the global budget bound the outer loop. Committed results are resumed idempotently from input/configuration digests and the recorded library state. Source persistence, execution, and isolation are separate interfaces; Docker is not required (Appendix A.4).

This loop provides a testable hypothesis for agentic design: observed failures enable revisions that a single completion cannot condition on. Extra calls alone do not establish an advantage. We therefore compare against both a single completion and independent sampling under matched budgets (Section 1.7).

1.4 Testing the Reward Function Itself

Independent task evidence. For mathematics, suite labels are derived from trusted answers and an independently implemented equivalence checker; for code, from trusted execution tests. The

suites include correct and incorrect responses, parser edge cases, and oracle-validated perturbations. Content correctness and instruction compliance are evaluated separately. A proposed perturbation is admitted only if the task oracle confirms the intended relation; agent-generated tests are supplemental diagnostics, not a source of ground truth.

Let P_τ^v contain strictly ordered response pairs (x^+, x^-) from suite v , where $x = (q, r, \hat{r}, z)$ and the task oracle prefers x^+ to x^- . Reward discrimination is

$$Q_{\text{rank}}^v(g) = \frac{1}{|P_\tau^v|} \sum_{(x^+, x^-) \in P_\tau^v} \mathbf{1}\{g_s(x^+) > g_s(x^-)\}, \quad (4)$$

where g_s denotes the scalar output. Ties and execution failures do not pass a strict comparison. For pairs E_τ^v whose quality should be unchanged under the task contract, we also measure

$$Q_{\text{inv}}^v(g) = \frac{1}{|E_\tau^v|} \sum_{(x, x') \in E_\tau^v} \mathbf{1}\{|g_s(x) - g_s(x')| \leq \epsilon_\tau\}. \quad (5)$$

The tolerance is fixed before comparison, and errors count as failures. Unchanged-quality examples may include equivalent numeric representations or permitted whitespace changes; strict comparisons include wrong-answer substitutions and confirmed code mutations. Constant rewards can pass invariance tests but fail discrimination, so neither metric is sufficient alone.

Acceptance and audit. We report a vector of execution success, ranking accuracy, invariance, constraint violations, and false acceptance/rejection rates at a development-fixed score threshold. Task-specific minimum coverage and acceptance thresholds are predefined on training/development tasks and shared by all methods. An artifact is registered only if its interface and required development criteria pass; this constitutes development validation, not proof of general correctness. The frozen final artifact is evaluated once on the sealed audit suite. Audit outcomes cannot guide repair, registration, or candidate selection. Split provenance, mutation families, and thresholds are detailed in Appendix A.

A strong objective verifier may already be an effective reward. Whenever it is available under the scoring contract, direct oracle reward is a required baseline. We do not create an advantage by withholding an otherwise available checker from competing methods. Construction is useful only if it improves measurable quality, coverage, or cost over such direct use; downstream RL is needed to determine whether better offline diagnostics improve policy learning.

1.5 Minimal Memory and Consistent RL Scoring

The runtime retains only an episode workspace and a persistent artifact library $\mathcal{F}$. Each library entry stores code and version, applicability, dependencies, calibration, development-test provenance, and a short validation report. Tags and text matching suffice for retrieval. Equivalent source/configuration hashes are deduplicated, revisions receive new versions, and previously accepted artifacts are immutable. Raw transcripts remain in the trace store rather than becoming an additional long-term memory system. Evaluation splits cannot share test-derived artifacts.

For a query q , the router selects or constructs one artifact g_q before scoring its rollout group $\{r_i\}_{i=1}^G$. All candidates use the same artifact version, scoring components, calibration, and equal-mean aggregation:

$$s_i = g_{q,s}(q, r_i, \hat{r}, z), \quad A_i = \frac{s_i - \bar{s}}{\text{std}(s) + \epsilon_{\text{adv}}}. \quad (6)$$

Zero-variance groups receive zero advantages. Candidate-caused failures, such as invalid code or a candidate exceeding its execution limit, follow the task contract; infrastructure/API failures are logged separately. With partial execution, a candidate uses the mean of its successful components and records their mask. Different masks can affect group comparability; coverage and missingness patterns must therefore be evaluated separately. If all components fail for a candidate, its score is invalid and the downstream group follows a predeclared retry/fallback or omission policy, rather than treating failure as zero. Function updates take effect only for a subsequent rollout batch; rewards are not recomputed midway through optimization of an existing batch.

1.6 Recorded Traces and Learning to Design Rewards

We record every observable construction event, including task and split IDs, the library snapshot, controller/model versions, tool arguments and results, source patches, test reports, finalization status,

tokens, latency, and cost. Brief explicit design notes can be logged, but the trace does not depend on access to hidden model reasoning. Failures and repairs are retained, and versioned observations support replay. Audit records are stored separately and excluded from the controller history.

To test whether Qwen3-8B can learn this capability, a stronger teacher collects construction traces on training tasks. Successful development-validated traces, including their failed intermediate attempts and subsequent repairs, supply supervised targets for tool selection, arguments, code edits, and final submission. Training applies next-token loss only to assistant actions and program outputs; tool observations are conditioning context, not prediction targets. Task families/templates and their derived examples are split before trace collection and library construction. No held-out-task artifacts or audit feedback enter student training.

The student is evaluated as a reward designer with the same APIs, initial library, and budget as the teacher. It is distinct from any Qwen3-8B policy trained downstream using the resulting rewards. Comparisons include the untrained student, final-program-only SFT, full-trace SFT, and the teacher. Primary outcomes are held-out reward quality, successful construction rate, repair after observed failures, and construction cost, rather than imitation of the teacher’s wording. This protocol tests learnability; no successful distillation result is assumed.

1.7 Comparisons That Isolate Agentic Construction

The primary baseline uses one chat completion to produce an executable reward artifact. It receives the same task pack, initial library, model cards, permitted scoring APIs, and labeled development examples as the agent; it receives no feedback from executing its own draft. Every submitted artifact is scored by the same external harness. We additionally compare budget-matched independent drafts selected by a fixed development evaluator, and multi-turn self-revision without execution observations. These separate adaptive use of feedback from increased sampling or generation budget.

Reward representation (single versus checklist) is crossed with construction strategy; checklist component evaluations count toward the same total budget. Module ablations remove execution feedback, persistent-library reuse, custom code construction, or neural scoring APIs one at a time. The no-execution-feedback condition also disables construction-time Bash and interactive scoring calls; removing only `test_reward` measures the value of standardized validation while leaving self-written execution feedback available. Fixed versus accumulating libraries are compared on identically ordered construction streams with separate, uncontaminated audit sets. All variants use the same resource accounting, failure handling, and final evaluation. We report per-task metrics and quality–cost tradeoffs; any superiority over single-turn design is an empirical question. Appendix A specifies the controls and trace protocol.

A Reward Construction Protocol and Trace Specification

The experiment manifest must fix test-set sizes, budgets, acceptance thresholds, model revisions, and training hyperparameters before comparisons. This proposed protocol has not yet been evaluated.

A.1 Stream Interface and Reward Representation

The core interface consumes an iterator of query records and yields construction results. A JSONL adapter supports append-only output; Parquet inputs can be streamed in batches and joined with a result sidebar by stable query IDs. Original inputs remain unchanged. Each record includes a query ID, query, task-profile reference, optional reference/metadata, and the configured reward mode. The profile supplies versioned task requirements and development tests, avoiding duplication in every row. Missing test evidence permits an unvalidated definition, not a validated-quality claim.

A result records input and configuration digests, initial library snapshot, job/attempt IDs, terminal status, stop reason, reward key, optional inline definition, validation/trace references, fallback attribution, and usage. Successful, unvalidated, failed, and skipped records are distinguished. Fallback availability is not counted as successful construction. Interrupted attempts are recorded separately and do not enter the completed-job set.

A reward definition contains its schema version, mode, component list, aggregation, and runtime contract. Each component has a criterion ID/description, function source as a string, entrypoint, and normalization. Single mode contains one component; checklist mode contains a bounded planned list whose successful scores are averaged with equal weights. Empty lists, non-finite scores, and unsupported aggregation options are invalid. Weight parameters and weight optimization are not part of this revision. The scoring runtime returns per-component statuses/scores and the aggregate. Normalization is declared and frozen; duplicate or overlapping criteria are checked because they can implicitly change emphasis despite equal averaging.

Inline storage writes the definition directly into each result. Reference storage writes it once to an optional library and stores its SHA-256 key in query records. Canonical serialization fixes encoding, key ordering, number representation, and source line endings. The key covers sources, composition, calibration, and dependency/API versions. Query inputs and validation reports are separate; report validity is bound to the reward key, task contract, suite, and runtime version. Changed components, normalization, or dependencies invalidate prior validation. Hash equality provides exact deduplication, not semantic similarity; task tags and applicability evidence support reuse. Exported wrapper source is derived from this authoritative definition and its pinned aggregation runtime.

The planned checklist and equal-mean rule are fixed for a query and its rollout group. Components execute independently; malformed code or a runtime failure in one component does not prevent the others from running. Successful finite scores form an explicit active mask. Partial execution returns their arithmetic mean, a partial status, and coverage; an empty mask returns a failed status with no aggregate. All valid zero scores yield a valid zero reward, not an execution failure. Quality validation is separate from operational partial success. Different active masks across candidates can affect reward comparability and must be reported and analyzed in downstream RL; a common-mask or coverage-gated variant would be an additional declared experiment, not the current default. Component diagnostics supplement evaluation of aggregate reward quality.

A.2 Tool Interfaces and Trusted State Transitions

The task pack, catalogue, and a small library index can be supplied directly as context. Dedicated task-analysis/model-inspection tools are unnecessary. File Read/Write/Edit and Bash are optional coding-agent adapters; a filesystem is not the persistent reward representation. Optional registry search accepts a query, resource kind, task tags, and limit, and returns versioned definitions and applicability/validation references. Search matches do not certify applicability.

Tool responses contain schema/call IDs, operation status, structured results/errors, usage, and a trace reference. Errors have stable categories, phases, and retryability. Negative reward-quality findings are valid test-tool observations, not transport errors. Truncation is marked and complete permitted observations remain accessible by reference. File adapters enforce version-aware edits; shell adapters

enforce the configured backend’s directory, time, and output limits. Both native and normalized events are retained, including source changes made through Bash.

Validation. `test_reward` accepts a serialized definition, an allowed development profile, a failure-display limit, and an on-pass policy (submit or return). A file adapter may first serialize a workspace. The supervisor creates an immutable artifact and calls the runner; profiles do not allow arbitrary evaluators or audit access. Reports identify the reward key, environment fingerprint, suite/profile, completed/errored counts, component and aggregate diagnostics, eligibility, and usage. Display limits do not change denominators. Missing metrics are null with a reason. The evaluator computes reports from actual scores and independent labels, rather than accepting agent-authored reports.

Submission. `submit_reward` identifies the selected key and matching complete development-validation run. The supervisor verifies ownership, applicability, versions, and acceptance rules. Submission ends the episode with a selected artifact; the outer driver commits storage. Old reports cannot validate changed definitions. Baselines without interactive tests submit their final definitions once for common external validation, with no feedback for further edits. Sealed audit remains a separate, non-interactive evaluator action.

The default efficiency policy allows a test action to request submission on success. The harness applies exactly the same finalizer checks, including full development validation, without a further assistant turn. Failure returns the report for repair; partial execution alone never implies admission. Explicit submission remains available for comparing candidates. A batch containing several candidates waits for the relevant reports and uses a predeclared selector, never arrival order. Submission policies are recorded across conditions; storage commit remains a separate driver responsibility.

A.3 Shared Prefixes and Efficient Controller Calls

One `llm_call` boundary assembles all controller requests. Inputs identify the episode, expected history cursor, immutable prefix/history references, model configuration, logical call ID, remaining budget, and deadline. Outputs include the complete observable assistant message and proposed actions, completion status, request digest, provider request ID, usage, and errors. The adapter validates schemas without an auxiliary repair model; the harness validates proposed actions before execution.

The logical context is a fixed run prefix, a frozen episode prefix, and append-only history. Instructions, tool definitions/order, and output schemas do not change within the episode. Task resources and the initial library snapshot are frozen; later drafts, observations, and budget updates appear only at the tail. Across queries, only identical permitted static resources share a prefix; conversations remain separate. The manifest pins the serializer/API adapter. Request metadata and retry timestamps stay outside the shared prompt prefix. Provider continuation IDs and prefix caches are optional optimizations, not the source of truth or a guarantee of identical wire formats or cache hits.

Within an episode, controller calls are serial. Independent tools may execute concurrently, but their results enter history in the original call order after all actions have outcomes or explicit terminal states. Dependent edits and reads execute sequentially; decisions requiring unseen results wait. The trace preserves actual completion times separately. One complete definition can contain the checklist plan and all component sources, and one test request can evaluate its components and development examples without separate model turns.

Ordinary code handles exact applicability checks, retrieval, hashing, report formatting, retry control, and finalization. Uncertain reuse is resolved by the same controller, not a hidden routing model. Reports use bounded deterministic diagnostics with references to complete permitted observations. Their visible representation is fixed on first insertion, with no later rewriting of history. Context preflight reserves space for output and bounded observations; overflow terminates with a recorded context-budget reason. This short-episode protocol uses neither silent sliding windows nor LLM summarization.

Checkpoints retain prefix/history digests, the committed message cursor, request references, adapter version, and pending batch order/status. Transport retries preserve the logical request; physical attempts and unknown usage remain separately accounted. A truncated response is diagnostic data, not an executable action; a further generation, if affordable, is a new logical request with an appended

diagnostic. Recovery reconstructs the same confirmed history without summary calls or duplicate messages. Late responses to a stale cursor cannot trigger actions.

Report controller decisions, physical API requests/retries, tool calls, neural scoring requests, input/output and cached tokens, cost, and latency separately. Batching reduces round trips, concurrency reduces waiting, and prefix caching can reduce prefill work; none demonstrates a quality or speed improvement without measurement. Single-completion and agentic conditions share initial information and final validation, while only the latter receives feedback for repair. Compare quality against total cost rather than assuming that additional or fewer calls establish superiority.

A.4 Execution Backends without a Docker Requirement

Construction, serialization, and stream control are independent of execution isolation. A runner accepts an immutable definition, permitted scoring inputs, capabilities, and resource limits, and returns structured results and usage. Suitable backends include an existing remote execution service, a configured Linux process sandbox, or an optional container/VM. The selected design retains generated Python functions. A restricted interpreter for predefined checker/API compositions would reduce this action space and is only an alternative, not the protocol adopted here. No particular service or backend has been deployed in this revision.

A local subprocess with a fixed environment is a lightweight prototype with crash and timeout management, not a filesystem/network security boundary. Generated Python is never imported into the main driver. Same-user subprocesses may access host files, so merely placing audit labels in another process or directory does not ensure isolation. Formal audits require an execution backend or permission boundary that enforces the declared separation. Bash is available only through the selected execution capability; omitting Docker does not grant unrestricted host-shell access.

The trusted evaluator retains oracle labels and service credentials outside the candidate’s accessible environment. Workers receive only permitted scoring inputs. Code candidates use separate bounded execution jobs, and scoring state is reset between examples. Model inference can remain in independent API services; worker CPU demand and isolation are separate issues. Limits cover wall time, memory, processes, outputs, and brokered requests, regardless of backend. Cold-start overhead may be amortized over a batch without sharing candidate execution state. Fixed dependencies and service versions permit replay without requiring a specific container image format.

A.5 Nested Termination, Errors, and Durable Results

Interruption and error-recovery rules. Recovery is defined at the synthesis-action level. Invalid tool arguments or reward-code/test errors are observations for agent repair. Transient transport/service faults are retried by the harness on the same immutable action input; an infrastructure outage is not labeled as poor reward quality and does not instruct the agent to rewrite the reward. Each error identifies its phase, action outcome, and retry owner (agent, harness, or none), preventing nested automatic and agent retries from multiplying indefinitely.

The checkpoint retains the last confirmed history, current definition/hash, eligible candidates, pending action/request IDs, versions, and consumed/reserved budget. A result already logged is consumed rather than re-executed. A dispatched request with unknown outcome is reconciled or safely retried with explicit duplicate-cost accounting. Incomplete streamed model output is diagnostic data, not a completed tool action. User/process interruption resumes from the last confirmed step only on continuation; budget exhaustion terminates the episode, and environmental/configuration errors remain distinct from semantic validation failure.

A query episode has finite limits on steps, revisions, tool/test/model calls, tokens, component count, and wall time. These are shared across checklist items and retries. The supervisor enforces transport/call timeouts and an independent episode deadline; checking a counter only before a potentially blocking call is insufficient. Repeated invalid actions or unchanged failing definitions trigger a bounded no-progress rule. Hard limits remain effective even if the agent produces superficially different drafts. Completed valid reuse or successful submission ends the episode; exhaustion selects a previously eligible artifact only under a predeclared rule, otherwise records failure.

Input/schema errors and repairable reward-code failures are contained within the affected query. Transient rate limits or runner failures receive bounded backoff charged to the same budgets. Authen-

tication/environment errors, failed durable writes, unknown driver faults, or a persistent infrastructure outage stop the outer run. A circuit breaker avoids spending the full dataset budget on a broken service. EOF, user cancellation, and global budget exhaustion stop intake and trigger bounded cleanup; they never silently mark unfinished jobs as complete. Every event carries an error category and termination reason.

The initial implementation can use a single writer with result/trace JSONL files, a run manifest, and an optional hash library. For a selected definition, persist the library record first, then the complete result record, then update the reconstructible index/checkpoint. The result record is the logical commit. An orphan library entry after a crash is not published for subsequent reuse; a committed result with a stale checkpoint is recovered from the result log. A partial final JSONL record is uncommitted. Concurrent multi-writer implementations require additional transaction/locking semantics.

Resume keys include stable input content, configuration, and the task’s recorded library snapshot. Committed records rebuild the library in the original stream order. Attempt IDs, consumed/reserved budgets, and external request IDs survive retries and restart; retries cannot reset all limits. Interrupted attempts remain resumable under explicit continuation. Logical result commits are idempotent, but external API calls are not assumed exactly-once: a request may have completed before a crash, so duplicate cost or unknown usage is logged unless the provider supplies idempotency guarantees.

A.6 Task Packs and Information Boundaries

Task packs record the task ID/subtype, contract, schemas, permitted resources, development suite, and oracle/test-builder versions. The harness maintains the sealed audit suite. Splits are created before generating response variants: all variants of one underlying question, code problem, or template stay in one split. Generalization claims require held-out task subtypes/templates, not only randomly held-out responses to familiar problems.

Table 2: Proposed objective tests for the initial mathematics and code scope. Mutations are retained only when the independent oracle confirms their labels.

Test purpose	Mathematics	Code
Discrimination	Correct vs. incorrect final answers under a trusted equivalence checker.	Passing vs. failing implementations under trusted task tests.
Content sensitivity	Change a sign, value, or unit when the task requires it; confirm the resulting error.	Introduce a boundary or logic error; confirm it is detected by independent tests.
Allowed invariance	Equivalent representations such as $1/2$ and 0.5 when both are allowed.	Semantics-preserving variable renaming or whitespace changes, checked by execution.
Constraint separation	Cross content correctness with satisfaction/violation of an explicitly required answer format.	Cross functional correctness with required interface/signature compliance.
Execution robustness	Empty output, malformed expressions, and multiple incompatible final answers.	Parse errors, invalid interfaces, nontermination, and task-specified resource violations.

Response pools are built before reward construction from trusted solutions, outputs of fixed candidate models, and validated mutations; their source and generation settings are recorded. Development and audit pools use disjoint base problems and include held-out mutation families for robustness analysis. The harness obtains quality labels before evaluating a candidate reward. For example, a code response that passes exposed examples but fails independent boundary tests is a useful negative for reward auditing. An independent checker does not call the generated parser to establish its own labels. A code oracle certifies behavior only on its declared tests, not program correctness on all possible inputs. Answer-equivalence tests certify final-answer correctness, not the validity of every reasoning step. These limits must accompany results.

The permitted scoring contract explicitly lists which reference answers, public tests, execution APIs, or checker APIs a reward may use. Stronger audit-only tests remain inaccessible to all methods, including the agent. A directly available exact checker is evaluated as an oracle-reward baseline. If direct verification is already sufficient, the study should report that boundary rather than claim an inherent benefit from construction.

Data roles. Training construction tasks generate teacher traces and reusable library entries. Development tasks tune controller prompts, acceptance rules, calibration, and resource budgets. Final held-out tasks have their own exposed development examples for constructing a program and a sealed audit partition for scoring that program. No audit observations enter construction. The distinction between a held-out task’s allowed development examples and its inaccessible audit labels is explicit in every event’s provenance.

For the primary held-out evaluation, every method starts from the same library built only from training tasks. Held-out tasks run in isolated workspaces and do not donate artifacts to other held-out tasks. A separate continual-reuse study may grow a library on an ordered construction stream, but uses disjoint audit queries, matched task order across methods, and repeated stream orders. That study is reported separately from unseen-task generalization.

A.7 Test Reports and Registration

Each test report contains the artifact hash, suite version, evaluated sample IDs, component scores, oracle comparisons, errors by cause, and aggregate metrics. Errors are retained in the relevant denominator. A failure to construct or finalize an eligible artifact counts against episode success; quality among successful artifacts is additionally reported but is not the sole metric. Pairwise accuracy alone does not determine admission: interface checks, score bounds, task-specific critical constraints, minimum test coverage, and predefined quality thresholds must also pass. Execution permits only declared resources and blocks audit access. Shared source screening rejects explicit example/answer lookup tables; held-out evaluation remains necessary to test generalization.

For binary correct/incorrect examples, fix a reward threshold b_τ using development data. False acceptance is the fraction of incorrect examples assigned $s \geq b_\tau$, and false rejection is the fraction of correct examples assigned $s < b_\tau$. Use all examples in the relevant class as the denominator; report execution errors separately because a missing score is neither a valid acceptance nor a valid rejection. Admission requires the execution criterion as well as these class diagnostics. Report pairwise discrimination, equality/near-equality behavior, and perturbation results independently of this threshold. The same thresholds and tie rules apply across compared methods; an absent test category is reported as unavailable, not as a perfect score.

Development-test queries and exposed failures are bounded. The final artifact is frozen before audit, and audit results never trigger a retry or replacement of that artifact. If construction fails, a downstream RL run may use the predeclared common fallback, but its use is logged and attributed to construction failure. Partial component execution follows the fixed successful-component mean; an all-component failure yields no score and triggers the downstream invalid-group policy in Section 1.5. Candidate timeouts defined as incorrect task behavior remain valid low-score outcomes.

A.8 Controlled Comparisons

Table 3: Construction controls and the question each comparison answers. All program outputs are evaluated by the same external harness.

Condition	Construction behavior	Purpose
Single completion	Generate one complete program from the shared task pack and catalogue.	Primary non-interactive reward-design baseline.
Independent drafts	Sample multiple programs without feeding scores back; a fixed selector chooses by development metrics.	Control additional sampling and selection under the same total budget.
Self-revision	Revise previous drafts for multiple turns without execution observations.	Isolate extra generation and reflection from test feedback.
Full agent	Inspect, retrieve, write, test, and repair adaptively.	Test whether observed failures improve subsequent designs.
Direct oracle	Use the permitted trusted verifier directly as the reward.	Determine whether reward construction is necessary for this task.
Fixed composition	Use a predefined rule/model combination with equal averaging.	Compare construction with a simple, reproducible alternative.

Single/checklist representation and single-completion/interactive construction are separate factors. Comparisons within each mode share scoring components and account for all component-level calls. All LLM construction conditions share the controller backbone/version, task descriptions, scoring resources, initial library, interface requirements, and development examples. For small catalogues and libraries, full descriptions can be included in every starting context. If retrieval is necessary, the same retrieval facility is exposed in every condition or its information is supplied in advance; only the adaptivity to observations differs. Task analysis must not silently use a stronger auxiliary model. Any learned helper is shared across conditions and its cost counted.

The primary single-completion comparison exposes the same labeled examples but provides no feedback about its generated program before submission. The budget-matched independent-drafts baseline uses the same development evaluator and a fixed, predeclared selection rule with critical-constraint checks and deterministic tie-breaking. Its entire generation and test cost is counted. A quality-cost curve at several common budget limits complements the single-completion result. Final external audit cost is reported separately and is the same measurement stage for all conditions.

Additional ablations have precise interventions: (i) disable construction-time execution, including Bash, candidate validation, and interactive scoring observations, while retaining identical public input files and final external measurement; (ii) remove the library mount and index from the controller while retaining the fixed primitive catalogue; (iii) disallow new custom verifier code and permit only selection or parameterized composition of fixed components; and (iv) remove neural scoring APIs while retaining code-based construction. Removing only `test_reward` while retaining Bash is a separate standardized-validator ablation: the agent can still write and run its own diagnostics. Model-API removal is enforced by the proxy, not merely by omitting its tool description. The custom-code and model-API ablations change available resources, so they measure component contributions rather than agentic control alone.

To isolate memory, compare a frozen initial library with an accumulating library on the same construction stream. Neither condition sees audit feedback. Report library hit rate, validated reuse rate, new artifact count, duplicate count, construction cost amortized over reuse, and reward quality. The frozen and growing conditions share the same neural API catalogue. Query routing can separately be compared with a task-metadata rule under the same frozen library.

Report per-task quality and construction success with uncertainty over task units and repeated construction seeds. Where the task mix supports a summary, use a declared task weighting rather than pooling all response pairs and letting large suites dominate. Downstream GRPO comparisons share policy initialization, rollout/training budget, fallback behavior, and held-out evaluation. Offline reward quality and downstream policy quality are separate endpoints.

A.9 Trace Format and Replay

An episode is an append-only sequence of events. Required fields are:

```
schema_version, episode_id, task_id, split, step,
actor_version, library_snapshot, tool_name, arguments,
observation_ref, observation_hash, artifact_before,
artifact_after, suite_version, status, usage, latency.
```

The manifest also records prompts, generation settings, dependencies, APIs, budget, seeds, final artifact, mode/checklist plan, and stop reason. Trace events include job/attempt and component IDs, retries, and budget consumption so nested loops can be reconstructed. Content-addressed source files, model cards, observations, and reports preserve the original trajectory. Replaying stored observations and re-executing versioned tools are separate reproducibility checks.

For controller calls, retain prefix/history digests, message cursor, actual request reference/digest, adapter version, logical call and physical attempt IDs, and batch call order. Stored model-visible messages preserve their original roles, IDs, and bounded observations. Automatic finalization is a harness event, not an invented assistant message.

Record visible actions, explicit design summaries, failed drafts, patches, and submissions without requiring hidden reasoning. Audit results are stored separately. Runtime retrieval returns reusable artifacts and short development reports, not prior conversation archives.

A.10 Proposed Qwen3-8B Distillation

Let $\mathcal{D}_{\text{trace}}$ contain successful, development-validated teacher episodes from training tasks. Every teacher attempt is retained in the raw archive; training selection rules are fixed on the training/development partitions and use no held-out audit outcomes. Intermediate failures within successful episodes remain in their causal order, so the student can learn to respond to actual tool errors and comparison failures. Fully failed episodes are used for failure analysis and may support later training variants; they are not silently relabeled as successful demonstrations.

For action tokens u in an assistant message or code submission, use

$$\mathcal{L}_{\text{SFT}}(\theta) = -\mathbb{E}_{\xi \sim \mathcal{D}_{\text{trace}}} \sum_{u \in \text{AssistantTokens}(\xi)} \log \pi_{\theta}(u \mid \xi_{<u}). \quad (7)$$

Tool observations condition subsequent actions but their tokens are loss-masked; automatic finalization events are not assistant targets. Full-trace examples retain the shared prefix and causal batch order and must fit the declared student context window. Overlength episodes are excluded with counts, or studied as a separately declared windowing variant that preserves failure–repair dependencies; they are not silently relabeled as full traces. The final-program-only control trains on the same task/artifact pairs with intermediate interaction removed; training-token budgets and data repetition are reported to distinguish interaction supervision from additional training compute.

Evaluate both SFT variants, the untrained student, and teacher under identical resources. Report construction success, sealed-audit quality, tool-schema validity, cost, and repair of standardized faulty drafts using real feedback. Hold out templates and mutation families to test generalization beyond program memorization. Student reinforcement learning is outside this revision.

B Historical Reward-Model Selection Study

Scope. This section preserves the selection-only study from the earlier manuscript. It does not evaluate the revised construction loop, its objective test suites, or student training. The reported numbers have not been recomputed for the revised protocol. The pool was drawn from a public leaderboard; label isolation and pool-selection effects require a separate audit.

B.1 Setup and Previously Reported Results

The earlier study evaluated reward-model selection on REWARDBENCH-V2. Selecting among fixed models is a narrower problem than constructing and testing an executable reward program.

Table 4: Historical comparison of agentic selection and reward merging on REWARDBENCH-V2.

Method	Avg	Precision IF	Math	Safety	Factuality	Focus
SOTA	87.19	57.14	60.00	97.12	85.29	99.13
<i>Agentic Selection within RLAR framework</i>						
Random	78.39	40.48	60.00	89.42	75.49	90.43
GPT-4o	88.19	57.14	68.57	96.15	87.25	99.13
GLM-4.6	88.69	57.14	68.57	96.15	88.24	100.0
GPT-5.1	90.44	66.67	71.42	96.15	90.19	100.0
<i>Logits Merging Methods</i>						
mean@2	87.44	61.90	68.57	95.19	83.33	99.13
mean@10	77.89	47.62	57.14	84.62	76.47	90.43
mean@50	51.51	40.48	48.57	66.35	48.04	46.09
Post-hoc Pool Oracle	93.47	76.19	77.14	97.12	95.10	100.0

Setup. We utilized a randomly and uniformly sampled subset of 400 instances from the REWARDBENCH-V2 test set. Each instance comprises one preferred (chosen) response and three non-preferred (rejected) responses for a given prompt.¹ For each instance, a reward model is required to score all four responses. An instance is considered a **pass** if the softmax-normalized reward score assigned to the preferred response exceeds a threshold of 0.5.

We leveraged the evaluation records of the top 50 classification models from the official leaderboard. RLAR is tasked with predicting the reward model most likely to pass a given instance, utilizing the query, candidate responses, and the models’ metadata. For evaluation simplicity, reward models are presented to the agent as encapsulated tools. We compared the following approaches: **Backbone LLM Scaling:** Evaluating the impact of different agentic capabilities using GPT-4o, GLM-4.6, and GPT-5.1. **Reward Logits Merging:** Assessing the effect of ensemble methods by calculating the average scores of the top- k models’ predictions on each candidate response (mean@ k).

Interpretation. These descriptive results compare selection and raw-logit merging within the reported pool. The post-hoc oracle uses evaluation outcomes and is not a deployable selector. Raw-logit averaging is sensitive to model-specific score scales. Neither the table nor model-card matching establishes the semantic correctness of synthesized reward programs; that requires the independent tests in Section 1.4.

¹The “Tie” category is excluded due to the test’s input-output format uniformity.